

Rapport Final

Securite NoSQL & DevSecOps

MongoDB RBAC • CI/CD Securise • SAST • AWS Terraform

Etudiant

Adam Mentagui

Formation

EIDIA — Cybersecurite

Periode

Fevrier – Avril 2026

Module 1 — NoSQL Security : MongoDB • Docker •
RBAC • Semgrep • SAST

Module 2 — DevSecOps : GitLab CI/CD • Gitleaks •
Terraform • AWS ECS

Table des matières

1	Introduction Generale	3
1.1	Structure des depots	3
2	Presentation des Technologies Utilisees	4
2.1	Technologies communes	4
2.2	Technologies – Module NoSQL Security	4
2.3	Technologies – Module DevSecOps	4
	PARTIE 1 – Module : Securite NoSQL	5
3	Description des TPs – Module NoSQL	6
3.1	TP : Securite MongoDB dans un SOC-lite	6
3.1.1	Objectif	6
3.1.2	Architecture du projet SOC-lite	6
3.1.3	Deploiement securise de MongoDB	6
3.1.4	Authentification MongoDB	6
3.1.5	Gestion des roles (RBAC)	7
3.1.6	Creation des utilisateurs	7
3.1.7	Connexion backend securisee	7
3.2	TP : Analyse SAST – secure-doc-manager	8
3.2.1	Objectif	8
3.2.2	Resultats njsscan	8
3.2.3	Vulnerabilites detectees et corrigees	8
4	Architecture des Environnements – NoSQL	10
4.1	Environnement de deploiement	10
4.2	Flux de securite	10
5	Mesures de Securite – NoSQL	11
5.1	Recapitulatif des utilisateurs et roles	11
5.2	Tests de securite RBAC	11
5.2.1	Test lecture seule	11
5.2.2	Test manager d’incidents	11
5.2.3	Test utilisateur applicatif	11
6	Vulnerabilites Etudiees – NoSQL	12
6.1	OWASP Top 10 – Couverture	12
7	Difficultes et Conclusion – NoSQL	13
7.1	Difficultes rencontrees	13
7.2	Conclusion	13
	PARTIE 2 – Module : DevSecOps	14

8	Architecture DevSecOps	15
8.1	Presentation du pipeline global	15
8.2	Philosophie DevSecOps – Shift Left	15
9	Description des TPs – DevSecOps	16
9.1	Pipeline 1 – Application upload-app (Seances 1–2)	16
9.1.1	Objectif	16
9.1.2	Architecture de l’application	16
9.1.3	Definition du pipeline – 5 stages	16
9.1.4	Fichier .gitlab-ci.yml	16
9.1.5	Dockerfile	18
9.1.6	Resultats du pipeline	18
9.1.7	Demarche pas a pas	19
9.2	Pipeline 2 – Deploiement AWS avec Terraform (Seances 3–4)	19
9.2.1	Objectif	19
9.2.2	Architecture cible AWS	19
9.2.3	Structure du depot GitLab	20
9.2.4	Configuration IAM	20
9.2.5	Infrastructure Terraform	20
9.2.6	Gestion de l’etat Terraform	20
9.2.7	Pipeline CI/CD Terraform	21
9.2.8	Variables sensibles dans GitLab CI/CD	21
9.2.9	Acces a l’application deployee	21
10	Analyse des Resultats des Scans – DevSecOps	22
10.1	Resultats Gitleaks	22
10.2	Resultats npm audit	22
10.3	Resultats Terraform plan/apply	22
11	Difficultes et Conclusion – DevSecOps	23
11.1	Difficultes rencontrees	23
11.2	Conclusion	23
12	Conclusion Generale	24

Chapitre 1

Introduction Generale

Ce rapport final regroupe l'ensemble des travaux realises durant le semestre dans le cadre de deux modules complementaires : **Securite NoSQL** et **DevSecOps**. Ces deux modules s'inscrivent dans une demarche globale de *Security by Design* visant a integrer la securite a chaque etape du cycle de developpement logiciel.

Critere	Module 1 – NoSQL Securite	Module 2 – DevSecOps
Depot GitHub	nosql-security-project	devsecops-project
Focus	Securite base de donnees	Pipelines CI/CD securises
Outils cles	MongoDB, Docker, RBAC	GitLab CI, Terraform, AWS
TPs couverts	Seances NoSQL (SAST, RBAC)	Seances 1-2, 3-4 (DevSecOps)

1.1 Structure des depots

nosql-security-project/

```
TP1/   TP2/   TP3/
TP4/   TP5/   TP6/   TP7/
docker/
screenshots/
docs/
rapport_final.pdf
docker-compose.yml
README.md
requirements.txt
```

devsecops-project/

```
Pipeline1/  Pipeline2/
Pipeline3/  Pipeline4/
Pipeline5/
scripts/    docker/
kubernetes/ screenshots/
docs/
rapport_final.pdf
.gitlab-ci.yml
docker-compose.yml  README.md
```

Chapitre 2

Presentation des Technologies Utilisees

2.1 Technologies communes

Technologie	Version	Role
Docker	24+	Conteneurisation des applications et services
Node.js	18 / 22	Runtime JavaScript cote serveur
GitLab CI/CD	—	Orchestration des pipelines DevSecOps
npm audit	v10	Audit des dependances Node.js

2.2 Technologies – Module NoSQL Security

Outil	Version	Role
MongoDB	7.0	Base de donnees NoSQL orientee documents
Semgrep	latest	Analyse statique (patterns nodejs, jwt, nosql)
njsscan	v0.4.3	Detection vulnerabilites Node.js
bcryptjs	—	Hachage mots de passe (12 rounds)
express-mongo-sanitize	—	Protection injections NoSQL
express-rate-limit	—	Rate limiting anti-bruteforce

2.3 Technologies – Module DevSecOps

Outil	Version	Role
Gitleaks	latest	Detection de secrets dans le code source
Terraform	v1.14.5	Infrastructure as Code (IaC)
AWS ECS Fargate	—	Orchestration de conteneurs sans serveur
AWS ECR	—	Registre d'images Docker prive
AWS ALB	—	Equilibrage de charge applicatif
AWS IAM	—	Gestion des identites et acces
Multer	—	Gestion des uploads de fichiers (Node.js)

PARTIE 1 — MODULE : SECURITE NoSQL

Chapitre 3

Description des TPs – Module NoSQL

Ce module couvre la securisation d'une base de donnees MongoDB deployee dans le contexte d'un SOC-lite (*Security Operations Center* simplifie), ainsi que l'analyse de securite statique (SAST) de l'application `secure-doc-manager`.

3.1 TP : Securite MongoDB dans un SOC-lite

3.1.1 Objectif

Securiser l'accès a une instance MongoDB en implementant l'authentification obligatoire, le controle d'accès base sur les roles (RBAC) et la separation des privileges.

3.1.2 Architecture du projet SOC-lite

L'application SOC-lite est organisee en deux parties : un backend Node.js/Express gerant la logique metier et la connexion MongoDB, et un frontend React/Vite pour l'interface utilisateur. La base MongoDB `soc_lite` contient trois collections :

Collection	Contenu
<code>security_logs</code>	Evenements de securite (logs bruts)
<code>incidents</code>	Incidents detectes et traites
<code>users</code>	Utilisateurs de l'application SOC

MongoDB est execute via Docker pour garantir un environnement isole et reproductible.

3.1.3 Deploiement securise de MongoDB

Par default, MongoDB n'impose aucune authentification. Le deploiement securise consiste a lancer le conteneur avec un compte administrateur et l'authentification activee implicitement. L'accès est restreint a `localhost` uniquement via le binding `127.0.0.1:27017`, rendant la base inaccessible depuis l'exterieur.

```
1 docker run -d --name mongo_secure \  
2   -p 127.0.0.1:27017:27017 \  
3   -e MONGO_INITDB_ROOT_USERNAME=adminSoc \  
4   -e MONGO_INITDB_ROOT_PASSWORD=Passw0rd_S3cure! \  
5   mongo:7
```

Listing 3.1 – Commande Docker – lancement securise

3.1.4 Authentification MongoDB

Sans credentials, toute tentative d'accès retourne une erreur `MongoServerError[Unauthorized]` : la commande `show dbs` est bloquee. La connexion correcte necessite de fournir le nom d'utilisateur, le mot de passe et la base d'authentification `admin`.

```
mongosh -u adminSoc -p PasswOrd_S3cure! --  
authenticationDatabase admin
```

Listing 3.2 – Connexion correcte avec authentification

Une fois authentifié, les bases de données sont listées normalement, confirmant le bon fonctionnement du mécanisme d'authentification.

3.1.5 Gestion des roles (RBAC)

Un rôle personnalisé `incidentManager` a été créé pour accorder des droits CRUD exclusivement sur la collection `incidents`, sans aucun accès aux autres collections. Ce rôle illustre le principe du moindre privilège.

```
1 db.createRole({  
2   role: "incidentManager",  
3   privileges: [  
4     {  
5       resource: { db: "soc_lite", collection: "incidents" },  
6       actions: ["find", "insert", "update", "remove"]  
7     }  
8   ],  
9   roles: []  
10  })
```

Listing 3.3 – Création du rôle `incidentManager`

3.1.6 Création des utilisateurs

Trois utilisateurs ont été créés avec des niveaux de privilèges distincts, assurant une séparation claire des responsabilités :

```
1 db.createUser({ user: "soc_app", pwd: "AppP@ss2026!",  
2   roles: [{ role: "readWrite", db: "soc_lite" }] })
```

Listing 3.4 – Utilisateur applicatif – `readWrite`

```
1 db.createUser({ user: "soc_analyst_ro", pwd: "AnalystR0@2026!"  
2   ,  
   roles: [{ role: "read", db: "soc_lite" }] })
```

Listing 3.5 – Utilisateur analyste – lecture seule

```
1 db.createUser({ user: "soc_manager", pwd: "Mgr$ecure2026!",  
2   roles: [{ role: "incidentManager", db: "soc_lite" }] })
```

Listing 3.6 – Manager incidents – rôle personnalisé

3.1.7 Connexion backend sécurisée

Le backend utilise l'utilisateur `soc_app` dédié, dont les credentials sont stockées dans un fichier `.env` exclu du dépôt via `.gitignore`. Le compte administrateur `adminSoc` n'est jamais utilisé par l'application.


```
1 MONGO_URI=mongodb://soc_app:AppP@ss2026!@127.0.0.1:27017/
   soc_lite?authSource=soc_lite
```

Listing 3.7 – Fichier .env – URI de connexion securisee

3.2 TP : Analyse SAST – secure-doc-manager

3.2.1 Objectif

Integrer des outils SAST (Semgrep, njsscan, npm audit) dans un pipeline GitLab CI/CD pour detecter et corriger les vulnerabilites de l'application `secure-doc-manager`.

3.2.2 Resultats njsscan

njsscan v0.4.3 a analyse le repertoire `src/` et conclut : **No issues found**. Les 12 correspondances de pattern et 11 de semantic grep detectees ne constituent pas des vulnerabilites exploitables dans ce contexte, grace aux protections mises en place (`bcryptjs`, `jsonwebtoken`, `express-mongo-sanitize`).

3.2.3 Vulnerabilites detectees et corrigees

Vulnerabilite	Severite	Outil	Statut
Injection NoSQL	CRITICAL	Semgrep	CORRIGEE
JWT secret faible	HIGH	njsscan	CORRIGEE
Credentials hardcodes	HIGH	Semgrep	CORRIGEE
Rate limiting absent	MEDIUM	njsscan	CORRIGEE

Injection NoSQL – correction

Sans validation des entrees, un attaquant peut injecter des operateurs MongoDB malveillants comme `{ $gt: '' }` pour contourner l'authentification ou exfiltrer des donnees. La correction consiste a sanitiser toutes les entrees avec `express-mongo-sanitize` et a valider le type et la longueur.

```
1 // AVANT (vulnerable)
2 const docs = await Document.find({ title: req.query.title });
3
4 // APRES (securise)
5 const title = mongoSanitize.sanitize(req.query.title);
6 if (typeof title !== 'string' || title.length > 200)
7   return res.status(400).json({ error: 'Titre invalide' });
8 const docs = await Document.find({ title: { $regex: title,
   $options: 'i' } });
```

Listing 3.8 – Avant/apres – injection NoSQL

JWT secret faible

Le secret JWT est stocke dans une variable d'environnement avec plus de 256 bits d'entropie, et masque dans les variables CI/CD GitLab.

Rate limiting

```
1 const limiter = rateLimit({ windowMs: 15*60*1000, max: 100 });  
2 app.use('/api/', limiter);
```

Listing 3.9 – Rate limiting – 100 req/15min

Chapitre 4

Architecture des Environnements – NoSQL

4.1 Environnement de déploiement

- **OS** : Windows 11 (développement local)
- **Docker** : MongoDB 7.0 en conteneur isolé, accessible uniquement via `localhost:27017`
- **Backend** : Express.js sur le port 5000, avec nodemon en développement
- **Frontend** : React/Vite, communique avec le backend via API REST

4.2 Flux de sécurité



Chaque couche applique ses propres mécanismes : JWT au niveau API, `express-mongo-sanitize` contre les injections, rate limiting sur `/api/`, et RBAC MongoDB pour cloisonner les accès aux données.

Chapitre 5

Mesures de Securite – NoSQL

5.1 Recapitulatif des utilisateurs et roles

Utilisateur	Role	Perimetre
soc_app	readWrite	Lecture + ecriture sur toute la base soc_lite
soc_analyst_ro	read	Lecture seule sur soc_lite
soc_manager	incidentManager	CRUD sur incidents uniquement
adminSoc	root (admin)	Administration MongoDB uniquement

5.2 Tests de securite RBAC

5.2.1 Test lecture seule

L'utilisateur `soc_analyst_ro` peut lire les collections de `soc_lite` normalement. Toute tentative d'ecriture (`insertOne`) ou de suppression (`deleteOne`) retourne une erreur `Unauthorized`, confirmant que le role `read` est bien applique.

5.2.2 Test manager d'incidents

L'utilisateur `soc_manager` dispose d'un acces CRUD complet sur la collection `incidents` : `find`, `insertOne`, `updateOne` et `deleteOne` fonctionnent. En revanche, toute tentative d'acces a `security_logs` ou `users` est refusee, validant le cloisonnement du role `incidentManager`.

5.2.3 Test utilisateur applicatif

L'utilisateur `soc_app` dispose des droits `readWrite` sur l'ensemble de la base. Les lectures et ecritures sur toutes les collections sont autorisees, ce qui correspond au besoin du backend applicatif.

Chapitre 6

Vulnerabilites Etudiees – NoSQL

6.1 OWASP Top 10 – Couverture

OWASP	Vulnerabilite	Couverture
A01 :2021	Broken Access Control	RBAC + ABAC
A02 :2021	Cryptographic Failures	JWT 256bit + bcrypt 12 rounds
A03 :2021	Injection (NoSQL)	express-mongo-sanitize
A05 :2021	Security Misconfiguration	MongoDB auth + Docker localhost-only
A06 :2021	Vulnerable Components	npm audit – 0 vulnerabilites
A07 :2021	Auth and Session Failures	JWT + rate limiting 100 req/15min

Chapitre 7

Difficultes et Conclusion – NoSQL

7.1 Difficultes rencontrees

- **Regle Semgrep p/nosql indisponible** : erreur HTTP 404 lors du telechargement ; les autres regles ont ete appliquees et la verification completee manuellement avec njsscan.
- **Variables CI/CD GitLab** : configuration des variables *masked* et *protected* pour eviter toute exposition de credentials dans les logs du pipeline.
- **Gestion du .gitignore** : s'assurer que `.env` n'est jamais commite dans le depot, en le validant localement avant chaque push.

7.2 Conclusion

Objectifs NoSQL atteints

- Authentification MongoDB activee – acces impossible sans credentials
- RBAC implemente avec 3 niveaux de privilege distincts
- Tests de securite valides pour les 3 profils utilisateur
- Backend connecte via utilisateur dedie, jamais via compte admin
- Pipeline SAST automatique integre dans GitLab CI/CD
- Zero vulnerabilite critique detectee par npm audit

Ameliorations futures : chiffrement TLS pour MongoDB, audit logs natifs, monitoring avec Prometheus/Grafana, rotation automatique des secrets via HashiCorp Vault.

PARTIE 2 — MODULE : DevSecOps

Chapitre 8

Architecture DevSecOps

8.1 Présentation du pipeline global

Le module DevSecOps couvre deux projets principaux implementes sur deux seances :

Seance	Projet	Objectif
Seances 1-2	Application upload-app	Pipeline CI/CD 5 stages avec Gitleaks et npm audit
Seances 3-4	Deploiement AWS avec Terraform	IaC sur ECS Fargate via GitLab CI/CD

8.2 Philosophie DevSecOps – Shift Left

Shift Left Security

Le DevSecOps integre la securite **des le debut** du pipeline, plutot qu'en phase finale. Chaque commit declenche automatiquement : detection de secrets (Gitleaks), audit des dependances (npm audit), construction Docker, et deploiement controle.

Chapitre 9

Description des TPs – DevSecOps

9.1 Pipeline 1 – Application upload-app (Seances 1–2)

9.1.1 Objectif

Mettre en place un pipeline CI/CD complet pour une application Node.js d'upload de fichiers, avec verifications de securite automatisees a chaque commit.

9.1.2 Architecture de l'application

L'application `upload-app` est une application Node.js/Express permettant aux utilisateurs de televerser, lister et supprimer des fichiers. Elle est organisee comme suit :

- **Backend** : Serveur Node.js/Express (`server.js`) avec Multer pour les uploads
- **Frontend** : Fichiers statiques dans `public/` (HTML, CSS, JS)
- **Uploads** : Dossier `uploads/` avec `.gitkeep` pour persistance Git
- **Tests** : Tests unitaires Jest via `supertest` dans `test.js`
- **CI/CD** : Pipeline GitLab avec `.gitlab-ci.yml` et conteneurisation via `Dockerfile`

9.1.3 Definition du pipeline – 5 stages

Le pipeline est compose de 5 stages sequentiels. Les stages `security` s'executent en parallele pour optimiser le temps d'execution. Le deploiement en production est manuel, necessitant une validation humaine avant mise en ligne.

Stage	Job	Description
build	build-job	npm install + verification syntaxique node -c
test	test-job	Tests unitaires Jest avec mesure de coverage
security	security-secrets-scan	Detection de secrets hardcodes avec Gitleaks
security	security-dependencies-scan	Audit des dependances npm (niveau high)
package	package-docker	Build image Docker et sauvegarde en artifact tar
deploy	deploy-staging	Deploiement automatique staging (port 3000)
deploy	deploy-production	Deploiement manuel production (port 8080)

9.1.4 Fichier `.gitlab-ci.yml`

```
1 stages:
```

```
2   - build
3   - test
4   - security
5   - package
6   - deploy
7
8   variables:
9     APP_NAME: "upload-app"
10    APP_VERSION: "1.0.0"
11
12   build-job:
13     stage: build
14     image: node:18
15     script:
16       - npm install
17       - node -c server.js
18     artifacts:
19       paths: [node_modules/]
20       expire_in: 1h
21
22   test-job:
23     stage: test
24     image: node:18
25     script: npm test
26
27   security-secrets-scan:
28     stage: security
29     image:
30       name: zricethezav/gitleaks:latest
31       entrypoint: [""]
32     script:
33       - gitleaks detect --source=. --verbose
34     allow_failure: false
35
36   security-dependencies-scan:
37     stage: security
38     image: node:18
39     script: npm audit --audit-level=high
40     allow_failure: false
41
42   package-docker:
43     stage: package
44     image: docker:24
45     services: [docker:24-dind]
46     script:
47       - docker build -t $APP_NAME:$APP_VERSION .
48       - docker save $APP_NAME:$APP_VERSION > $APP_NAME.tar
49     artifacts:
50       paths: [$APP_NAME.tar]
51       expire_in: 1h
52     only: [main]
```

```
53
54 deploy-staging:
55   stage: deploy
56   image: docker:24
57   services: [docker:24-dind]
58   before_script: [docker load < $APP_NAME.tar]
59   script:
60     - docker stop upload-app-staging || true
61     - docker rm upload-app-staging || true
62     - docker run -d --name upload-app-staging -p 3000:3000
        $APP_NAME:$APP_VERSION
63   environment:
64     name: staging
65     url: http://staging.upload-app.local:3000
66     only: [main]
67
68 deploy-production:
69   stage: deploy
70   image: docker:24
71   services: [docker:24-dind]
72   script:
73     - docker run -d --name upload-app-prod -p 8080:3000
        $APP_NAME:$APP_VERSION
74   environment:
75     name: production
76     url: http://www.upload-app.com
77   when: manual
78   only: [main]
```

Listing 9.1 – Pipeline .gitlab-ci.yml complet

9.1.5 Dockerfile

```
1 FROM node:22.20.0
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm install
5 COPY . .
6 RUN mkdir -p uploads
7 VOLUME ["/app/uploads"]
8 EXPOSE 3000
9 CMD ["node", "server.js"]
```

Listing 9.2 – Dockerfile de l'application

9.1.6 Resultats du pipeline

Le pipeline s'est execute avec succes, tous les stages passant au vert. Le job `security-secrets-scan` a correctement detecte un secret de test (`API_KEY`) lors d'un commit deliberement malveillant, bloquant le pipeline. Apres suppression du fichier incrimine et nouveau push, le pipeline est passe integralement. La construction de l'image Docker s'est effectuee en 10 etapes, incluant la mise en cache des layers pour acclereler les builds

suivants.

9.1.7 Demarche pas a pas

1. Initialisation du projet

```
mkdir upload-app && cd upload-app
npm init -y && npm install express multer
npm install --save-dev jest supertest
```

2. **Creation du serveur** : configuration de Multer pour les uploads, routes GET (liste des fichiers), POST (upload), DELETE (suppression), et dossier uploads/ pour le stockage persistant.

3. Tests unitaires

```
test('upload de fichier', async () => {
  const response = await request(app)
    .post('/upload').attach('file', 'test.txt');
  expect(response.statusCode).toBe(200);
});
```

4. **Test de detection de secrets** : ajout delibere d'un fichier contenant une cle API pour valider Gitleaks, puis suppression et correction.

```
echo "const API_KEY = 'sk-test123456789';" > secret.js
git add secret.js && git commit -m "test secret" && git push
# Pipeline bloque par Gitleaks
git rm secret.js
git commit -m "fix: suppression du secret" && git push
```

5. **Correction et deploiement** : apres correction, le pipeline passe integralement et le deploiement staging se declenche automatiquement.

9.2 Pipeline 2 – Deploiement AWS avec Terraform (Seances 3–4)

9.2.1 Objectif

Deployer l'application `upload-app` sur AWS en utilisant Terraform pour l'Infrastructure as Code et GitLab CI/CD pour l'automatisation du deploiement.

9.2.2 Architecture cible AWS

L'environnement de production est entierement provisionne via Terraform et s'appuie sur les services AWS suivants :

- **Amazon ECS Fargate** : execution des conteneurs sans gestion de serveurs ; la task definition specifie les ressources CPU/memoire et l'image ECR a utiliser
- **Application Load Balancer (ALB)** : distribution du trafic entrant sur le port 80 vers les conteneurs ECS sur le port 3000
- **Amazon ECR** : stockage prive des images Docker, alimentes par le pipeline
- **Amazon VPC** : reseau isole avec 2 sous-reseaux publics dans des zones de disponibilite differentes, passerelle Internet et table de routage

- **Groupes de securite** : pare-feu limitant le trafic entrant aux ports 80 (ALB) et 3000 (conteneurs ECS)
- **AWS IAM** : role d'execution ECS autorisant l'accès à ECR et CloudWatch
- **CloudWatch** : groupe de logs centralisant les sorties des conteneurs

9.2.3 Structure du depot GitLab

Le depot `upload-app-final` separe clairement le code applicatif de l'infrastructure :

```
1 upload-app-final/  
2   .gitlab-ci.yml      # Definition de la pipeline  
3   app/                # Code source Node.js  
4     server.js  
5     Dockerfile  
6     ...  
7   terraform/          # Code d'infrastructure  
8     main.tf  
9     variables.tf  
10    outputs.tf
```

Listing 9.3 – Structure du depot GitLab

9.2.4 Configuration IAM

Un utilisateur IAM `adam` a ete cree dans la console AWS avec les politiques necessaires. Des cles d'accès ont ete generees et enregistrees localement via `aws configure`. Ces cles sont ensuite reportees dans les variables CI/CD GitLab comme variables *masked* et *protected*.

9.2.5 Infrastructure Terraform

Le fichier `main.tf` provisionne l'ensemble des ressources AWS : VPC avec 2 sous-reseaux publics, passerelle Internet, groupes de securite (ports 80 et 3000), depot ECR, cluster ECS avec task definition et service, ALB avec target group et listener sur le port 80, et groupe de logs CloudWatch.

```
1 terraform {  
2   backend "http" {  
3     # Etat gere automatiquement par GitLab CI/CD  
4   }  
5 }
```

Listing 9.4 – Configuration backend HTTP dans `main.tf`

9.2.6 Gestion de l'etat Terraform

La gestion de l'etat `terraform.tfstate` est critique en contexte CI/CD. Trois solutions ont ete evaluees :

- **Backend local** : inadequat, l'etat n'est pas partage entre les runners
- **Backend S3** : robuste mais necessite la configuration d'un bucket S3 et d'une table DynamoDB pour le verrouillage
- **Backend HTTP GitLab** : solution retenue, native a la plateforme, ne necessite aucune ressource AWS supplementaire

9.2.7 Pipeline CI/CD Terraform

```
1 stages:
2   - validate
3   - plan
4   - apply
5
6 validate:
7   stage: validate
8   script: gitlab-terraform validate
9
10 plan:
11   stage: plan
12   script:
13     - gitlab-terraform plan
14     - gitlab-terraform plan-json
15
16 apply:
17   stage: apply
18   script: gitlab-terraform apply
19   when: manual    # Validation manuelle avant deploiement
```

Listing 9.5 – Pipeline GitLab pour Terraform

Les stages `validate` et `plan` s'exécutent automatiquement à chaque push, tandis que `apply` nécessite une validation manuelle pour éviter tout déploiement non intentionnel en production.

9.2.8 Variables sensibles dans GitLab CI/CD

Les trois variables AWS sont configurées dans les paramètres CI/CD de GitLab avec les flags *masked* (valeur invisible dans les logs) et *protected* (disponible uniquement sur les branches protégées) : `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_DEFAULT_REGION`.

9.2.9 Accès à l'application déployée

L'application est déployée derrière un ALB, donc inaccessible via IP directe. L'URL publique est au format `http://[nom-dns-alb].eu-west-3.elb.amazonaws.com`, récupérable via la console AWS (EC2 → Load Balancers) ou via les outputs Terraform après ajout de la variable `alb_url` dans `outputs.tf`.

Chapitre 10

Analyse des Resultats des Scans – Dev-SecOps

10.1 Resultats Gitleaks

Gitleaks a detecte et bloque le commit contenant la cle API `sk-test123456789`, retournant une erreur qui stoppe le pipeline avec `allow_failure: false`. Ce comportement valide le bon fonctionnement du scan de secrets. Apres suppression du fichier `secret.js` et nouveau push, le pipeline reprend normalement.

10.2 Resultats npm audit

Aucune vulnerabilite de niveau *high* ou *critical* n'a ete detectee dans les 398 packages analyses. Des warnings de deprecation apparaissent pour `supertest`, `superagent` et `glob`, mais ces packages ne presentent pas de risques de securite immediats et ne bloquent pas le pipeline (seuil configure a `-audit-level=high`).

10.3 Resultats Terraform plan/apply

La pipeline Terraform s'est executee avec succes. La phase `plan` a liste l'ensemble des ressources a creer (VPC, subnets, ECR, ECS, ALB, IAM, CloudWatch). La phase `apply`, declenchee manuellement, a provisionne l'infrastructure en quelques minutes. Des warnings mineurs ont ete emis (non bloquants), principalement relatifs a des attributs deprecies dans les providers AWS.

Chapitre 11

Difficultes et Conclusion – DevSecOps

11.1 Difficultes rencontrees

Probleme	Erreur	Solution
Image Gitleaks Tag Node.js	unknown command "sh" node:v22.20.0: not found	Ajouter entryptpoint: [""] Corriger en node:22.20.0
Image Docker	pull access denied	Utiliser docker save/load avec artifacts
Dossier uploads	ENOENT: no such file	RUN mkdir -p uploads dans Dockerfile
WSL2/Docker	Virtualisation inactive	Activer WSL2 et Hyper-V sous Windows
Sous-module Git	App comme sous-module	Supprimer de l'index, rajouter en dossier standard
Duplicate providers	Syntaxe Terraform	Fusionner les blocs required_providers
Limite VPCs AWS	Quota VPC depasse	Nettoyage des VPCs inutilises via console AWS
Stale plan	Backend HTTP GitLab	Relancer plan avant chaque apply

11.2 Conclusion

Objectifs DevSecOps atteints

- Pipeline CI/CD 5 stages operationnel – tous les jobs au vert
- Detection de secrets automatisee avec Gitleaks (Shift Left)
- Conteneurisation Docker avec volume persistant pour les uploads
- Deploiement staging automatique / production manuelle
- Infrastructure AWS entierement provisionnee via Terraform (IaC)
- Etat Terraform gere de facon securisee via backend HTTP GitLab
- Variables sensibles AWS protegees (masked + protected dans GitLab)

Ameliorations futures : certificat SSL via AWS Certificate Manager, environnements multiples (dev/test/prod), base de donnees Amazon RDS, domaine personnalise via Route 53, metriques CloudWatch avancees, authentication OIDC GitLab-AWS (alternative aux clefs IAM statiques).

Chapitre 12

Conclusion Generale

Ce rapport demontre la mise en oeuvre d'une demarche de securite complete et integree, couvrant deux dimensions complementaires du developpement securise moderne.

Le **Module NoSQL** a etabli des fondations solides pour la securite des donnees : authentication obligatoire, controle d'accès granulaire via RBAC, et detection proactive des vulnerabilites par analyse statique. Chaque composant du systeme respecte le principe du moindre privilege.

Le **Module DevSecOps** a demontre l'automatisation de la securite dans le cycle de developpement : chaque commit est systematiquement analyse, les secrets sont detectes avant d'atteindre le depot, et l'infrastructure est deployee de facon reproductible et tracable via Terraform.

Security by Design

La securite n'est pas une fonctionnalite a ajouter en fin de projet, mais une discipline a integrer a chaque etape du developpement.

*Rapport redige par **Adam Mentagui***

EIDIA Cybersecurite – Année universitaire 2025–2026